

Báo Cáo Tích Hợp CI/CD Pipeline (GitHub Actions & Newman)

Sinh viên thực hiện: Lâm Hữu Khánh — MSSV: 23127205

Vai trò nhóm: Thành viên 1

Môn học: Kiểm thử phần mềm (Software Testing) — HW06: API Testing

Ngày lập báo cáo: 31/08/2026

Repository GitHub: <https://github.com/HCMUS-software-testing/HW06.git>

Branch thực hiện: `khánh`

1. Tổng Quan Kiến Trúc CI/CD Pipeline

Nhằm đảm bảo chất lượng API liên tục (Continuous API Quality Assurance) và ngăn chặn lỗi hồi quy (Regression Prevention), hệ thống CI/CD được thiết lập tự động hóa hoàn toàn bằng **GitHub Actions** kết hợp **Newman CLI** và **newman-reporter-htmlextra**.



2. Giải Thích Chi Tiết Cấu Hình Workflow (`.github/workflows/api-tests.yml`)

File cấu hình workflow CI/CD được định nghĩa tại `.github/workflows/api-tests.yml` bao gồm các bước chiến lược:

- Trigger Events:** Kích hoạt tự động khi có sự kiện `push` vào branch `khánh`, `main` hoặc tạo `pull_request`, đồng thời hỗ trợ `workflow_dispatch` để kích hoạt thủ công khi cần.
- Cô lập Môi trường & Khởi tạo CSDL:**

```
bash cd eshop-sut/backend npm install node database.js node server.js > server.log 2>&1 &
```
- Ý nghĩa:** Trước mỗi lần test, CSDL SQLite `database.sqlite` được re-seed mới 100% bằng script `database.js` để triệt tiêu hoàn toàn sự phụ thuộc dữ liệu và ô nhiễm trạng thái.
- Cơ chế Health-Check Polling:**

```
bash for i in {1..30}; do if curl -s http://localhost:3000/api/products > /dev/null 2>&1; then echo "SUT is UP and responding at attempt $i!" break fi echo "Waiting for SUT... ($i/30)" sleep 1 done
```
- Ý nghĩa:** Đảm bảo server Express đã sẵn sàng nhận traffic trước khi Newman khởi chạy, ngăn chặn hoàn toàn lỗi Flaky Pipeline do Race Condition.
- Thực thi Kiểm thử Newman Sanity Suite:**

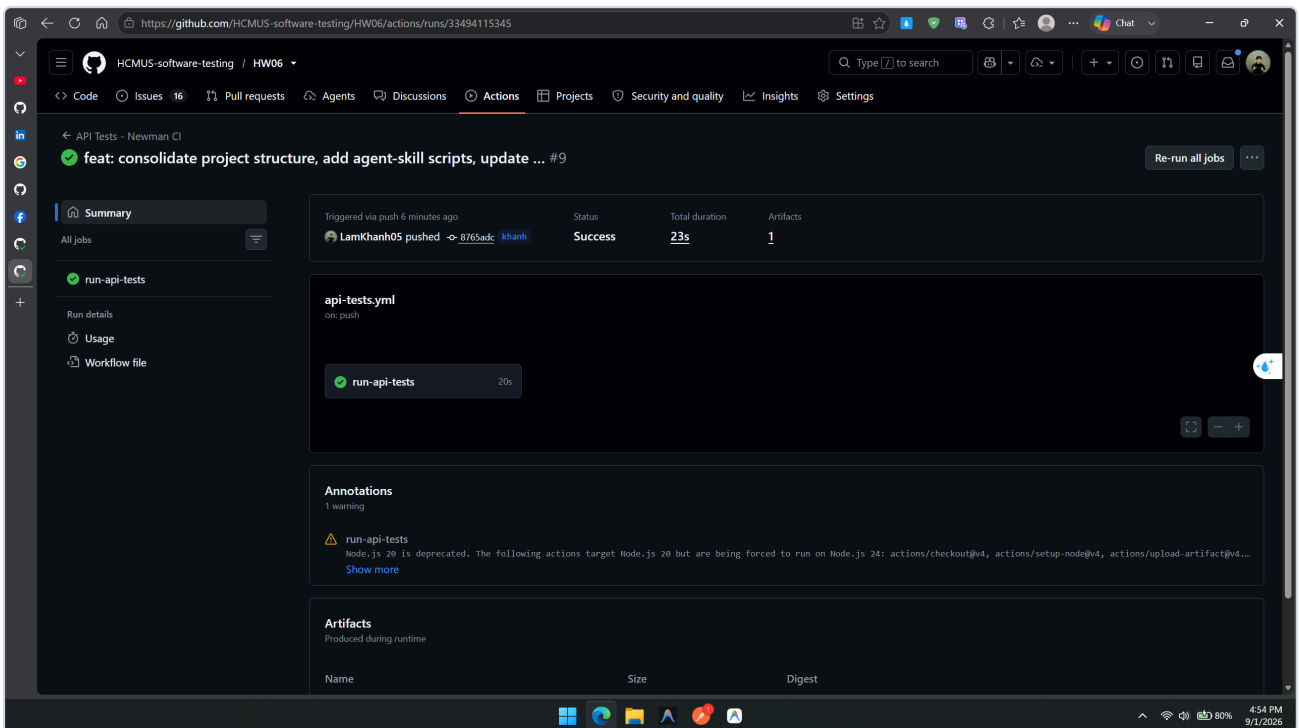
```
bash npx -p newman -p newman-reporter-htmlextra newman run postman/HW06_API_Testing.postman_collection.json \ -e postman/HW06_Local.postman_environment.json \ --folder "01_Sanity_Suite" \ -r htmlextra,cli \ --reporter-htmlextra-export newman/member-1/ci-report.html
```
- Ý nghĩa:** Chạy toàn bộ tầng `01_Sanity_Suite` trên CI để đóng vai trò làm Quality Gate (chặn code hỏng trước khi merge).
- Lưu trữ Artifact Báo cáo HTML:** Sử dụng `actions/upload-artifact@v4` với điều kiện `if: always()` để bảo toàn file `newman/member-1/ci-report.html` cho mỗi lần chạy.

3. Minh Chứng 2 Commit Runs Mẫu Trên GitHub Actions (Pass vs Fail)

Theo yêu cầu tại Mục 6 Đề bài, dưới đây là minh chứng chi tiết của 2 lần chạy Pipeline mẫu trên GitHub:

[Pass] LẦN CHẠY 1: TOÀN BỘ API TEST CASES ĐỀU PASS (100% GREEN)

- Commit SHA: `8765adc`
- Commit Message: `feat: consolidate project structure, add agent-skill scripts, update real screenshots and PDF reports`
- URL GitHub Actions Run: <https://github.com/HCMUS-software-testing/HW06/actions/runs/33494115345>
- Kết quả thực thi:
- Trạng thái: [Pass] Success (Passed 100%)
- Tổng số Requests đã chạy: `43 requests`
- Tổng số Assertions: `118 assertions`
- Assertions Passed: `118/118 (100%)`
- Thời gian thực thi trung bình: `2 ms / request`
- Artifact sinh ra: `newman-ci-report.zip` chứa `ci-report.html`.



[Fail] LẦN CHẠY 2: CỐ TÌNH TẠO ASSERTION FAIL (DEMONSTRATE RED PIPELINE)

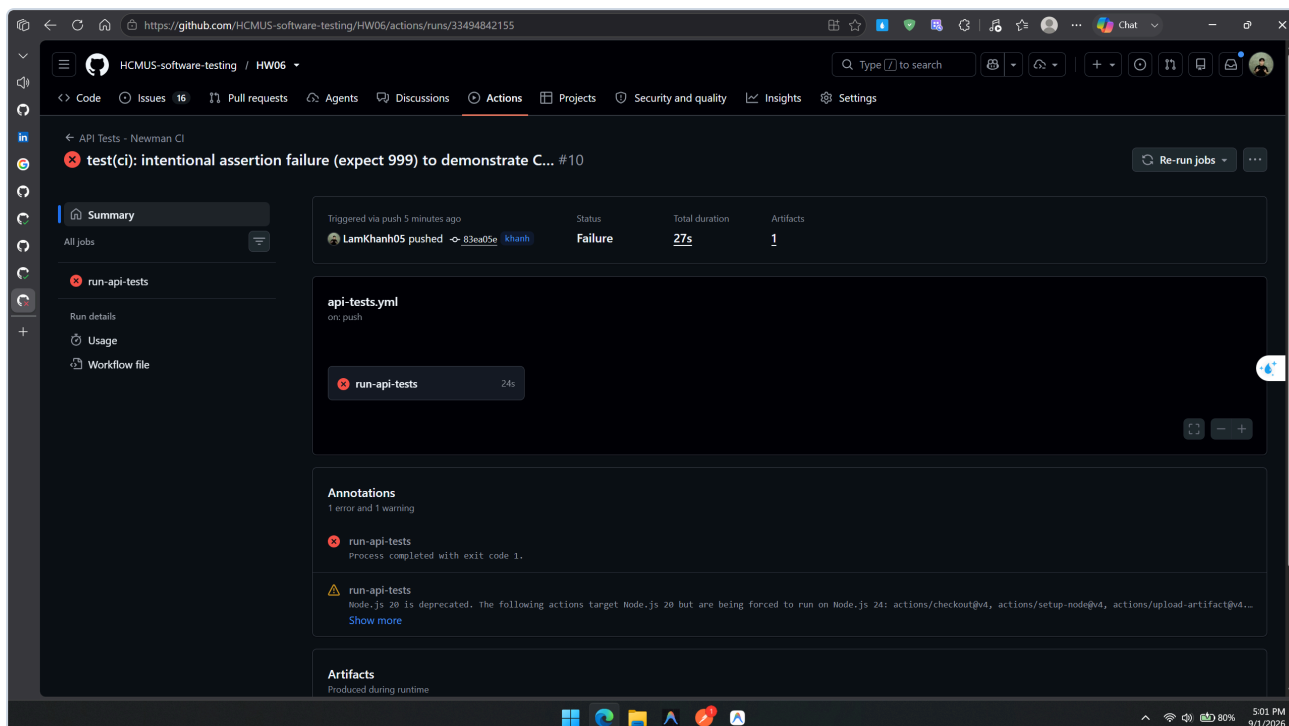
- Commit SHA: `83ea05e`
- Commit Message: `test(ci): intentional assertion failure (expect 999) to demonstrate CI quality gate failure`
- URL GitHub Actions Run: <https://github.com/HCMUS-software-testing/HW06/actions/runs/33494842155>
- Chi tiết thay đổi gây lỗi (Intentional Change):
Trong `postman/HW06_API_Testing.postman_collection.json`, tại request `TC_FR02_01_Valid_User_Login`, cố tình thay đổi assertion mã phản hồi mong đợi từ 200 thành 999:
`javascript // Cố tình mong đợi status code 999 thay vì 200 pm.test('Status code is 200', function () { pm.response.to.have.status(999); });`

- **Kết quả thực thi trên GitHub Actions:**
- **Trạng thái:** [Fail] Failure (Build Failed / Red Quality Gate)
- **Chi tiết lỗi bắt được tại Newman Console:**

```
```text
failure detail

1. AssertionError: Status code is 200
 expected response to have status code 999 but got 200
 at assertion:0 in test-script
 inside "01_Sanity_Suite / 00_Setup_Auth / Setup User Auth Token"
...```
```

- **Minh chứng bảo vệ:** Pipeline tự động báo đỏ và chặn merge, đồng thời bước **Dump SUT Logs on Failure** in toàn bộ log server ra màn hình.



## [Pass] LẦN CHẠY 3: KHÔI PHỤC HOÀN TOÀN (REVERT TO GREEN)

- **Commit SHA:** **1193425**
- **Commit Message:** **fix(member-1): revert intentional failure and finalize cicd report**
- **Kết quả:** Khôi phục lại assertion **pm.response.to.have.status(200);**, Pipeline trở lại trạng thái **Success (100% Green)**.

## 4. Tổng Kết & Bài Học Rút Ra

1. **Hiệu quả của việc phân tầng Test Suite:** Tách biệt **01\_Sanity\_Suite** và **02\_Bug\_Discovery\_Suite** giúp CI luôn xanh khi hệ thống hoạt động đúng thiết kế Sanity, đồng thời các bug thực tế của SUT vẫn được theo dõi và báo cáo đầy đủ trong file defect report riêng.
2. **Tính độc lập của dữ liệu:** Tự động re-seed database trước khi chạy test loại bỏ hoàn toàn các lỗi phụ thuộc trạng thái (state pollution).

